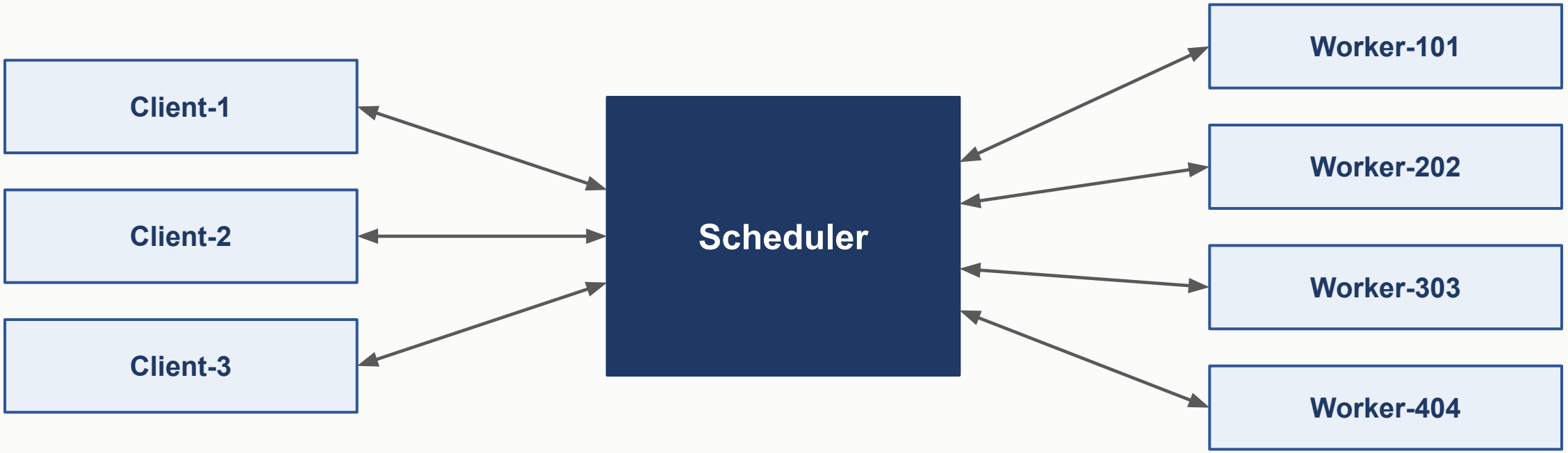
Distributed Compute Cluster

OSN Course Project

What will you build?

A small cloud. A user hands the cluster a program (a job), the cluster picks a machine, runs it safely and sends the output back.

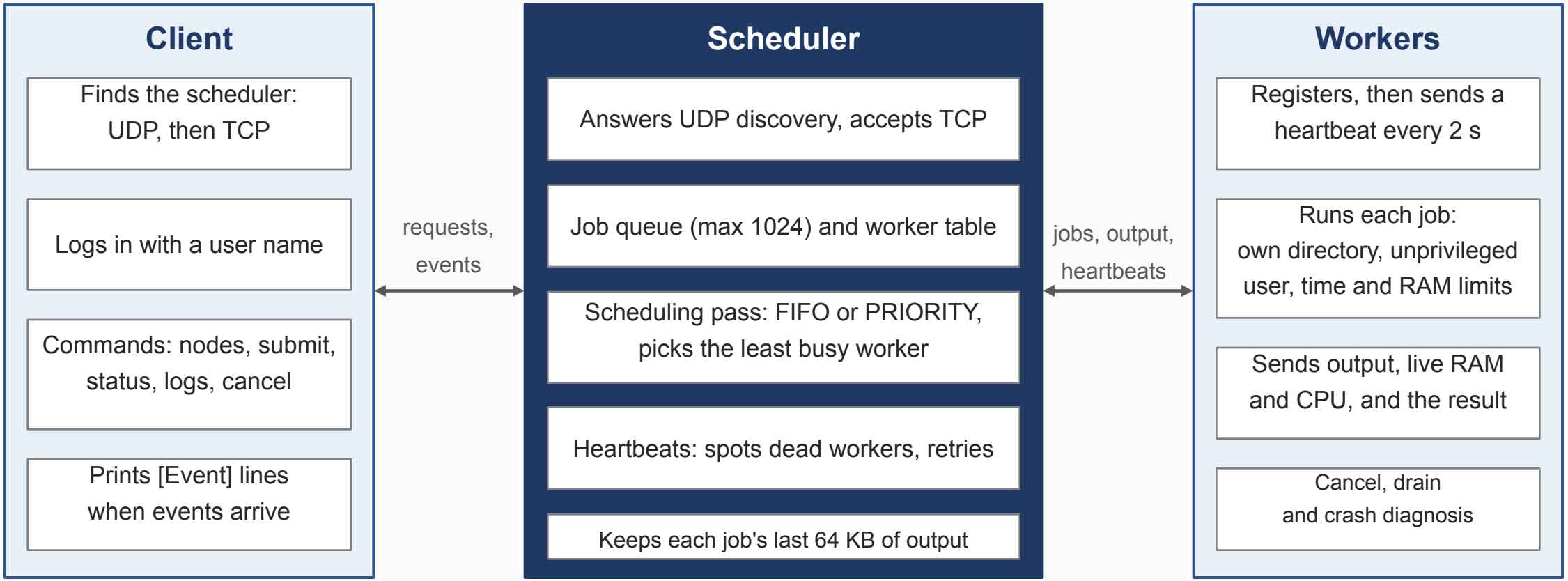


Client: the program where the user types commands

Scheduler: the central server that decides which worker runs each job

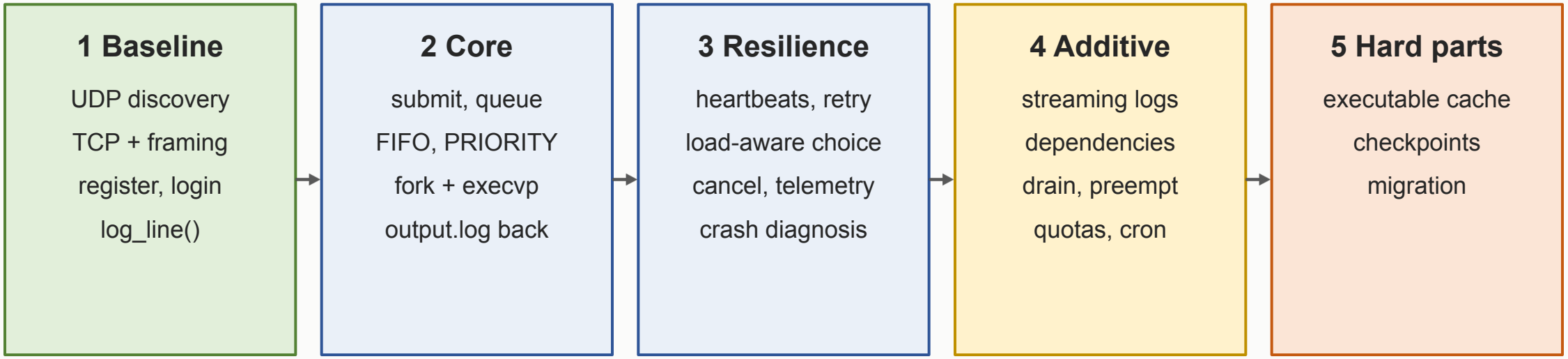
Worker: a machine that actually runs the jobs it is given

The whole system at a glance



Machines find each other with a UDP broadcast, and everything after that runs over TCP.

In what order to build it



Note: It is not as big as the PDF looks. Boxes 1 to 3 are the real system. Box 4 is a set of small, independent features that each hook into one spot of the scheduling pass or a message handler. Box 5 is where the real difficulty is.

The additive jobs

Once the core system works, each of these is a small, independent addition.

Streaming logs

logs -f shows output while the job is still running

Logging

one log function with timestamps for every state change

Dependencies, groups

a job waits until the jobs it depends on have completed

Graceful draining

a worker finishes its jobs, then leaves cleanly

Preemptible jobs

a priority 1 job can kick out a preemptible one

Fair-share quotas

no user holds more than half the cluster while others wait

Cron jobs

-every N makes a fresh copy of the job on a timer

Command lines and what they print

Scheduler

```
./scheduler <port> FIFO|PRIORITY
  [-discovery-port <port>]

$ ./scheduler 8080 PRIORITY
[SCHEDULER] Started on port 8080.
  Algorithm: PRIORITY
[SCHEDULER] Discovery responder
  listening on UDP port 9999.
[SCHEDULER] Listening for incoming connections...
```

Client

```
./client [scheduler_ip port]
  [-user NAME]
  [-discovery-port <port>]

$ ./client
[Client] Connected to Scheduler.
Username: lordz
Logged in as lordz.

client> nodes
```

Worker (run it with sudo)

```
./worker [scheduler_ip scheduler_port] -cores <num> -ram <MB> [-gpu <num>] [-discovery-port <port>]

$ sudo ./worker -cores 4 -ram 2048 -gpu 1
[WORKER] Looking for scheduler via UDP discovery (port 9999)...
[WORKER] Scheduler found at 192.168.1.20:8080.
[WORKER 1001] Registered successfully. ID: 1001. Capacity: 4 Cores, 2048MB RAM, 1 GPU.
[WORKER 1001] Heartbeat active. Waiting for jobs...
```

The ip and port are only a fallback when discovery gets no reply. -cores and -ram are clamped to the real machine, with a warning.

Each part in detail

Who does what (Section 1)

Client

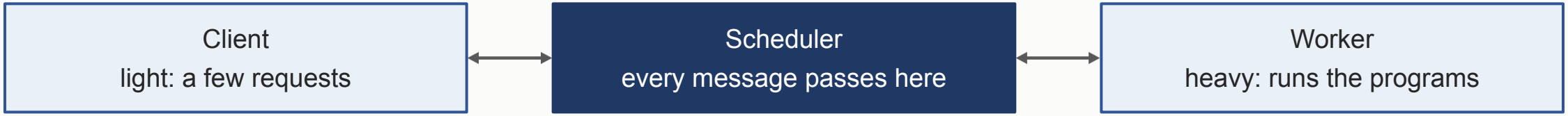
- Reads stdin and the socket at once
- Parses commands, sends requests
- Prints [Event] lines when events arrive

Scheduler

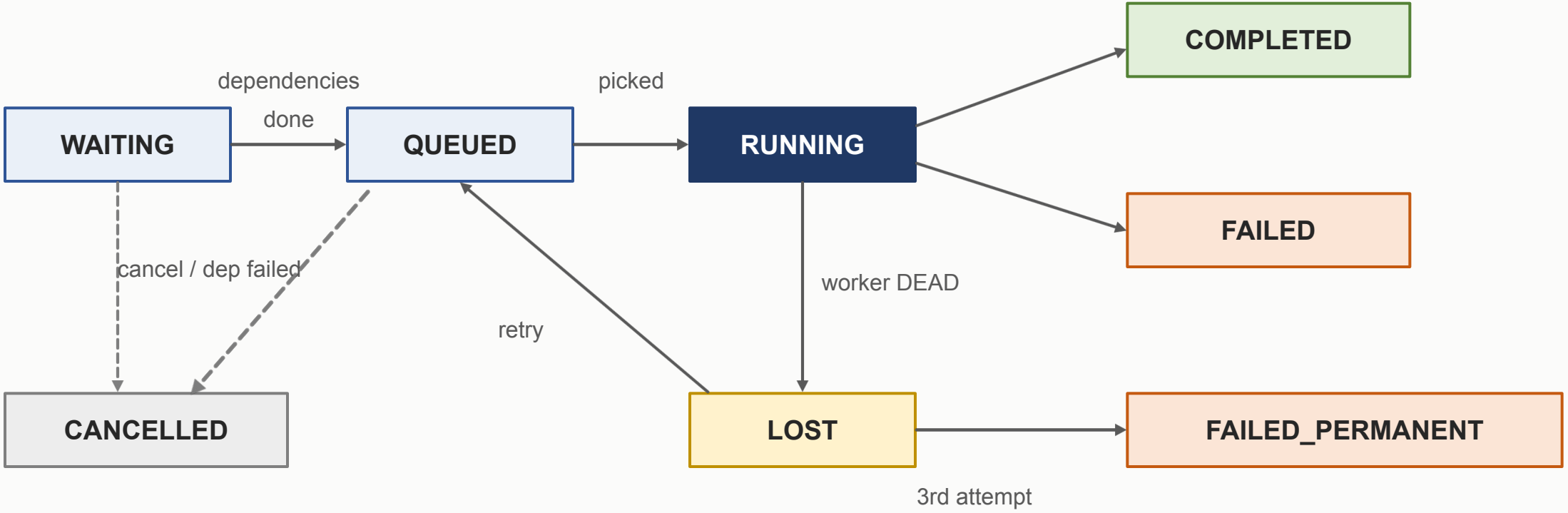
- Owns all state: queue, worker table, sessions
- One select() loop, checks workers every second
- Decides which worker runs which job
- Relays output and events

Worker

- Advertises its capacity
- Runs jobs with limits
- Enforces time and memory limits
- Sends heartbeats, output, results



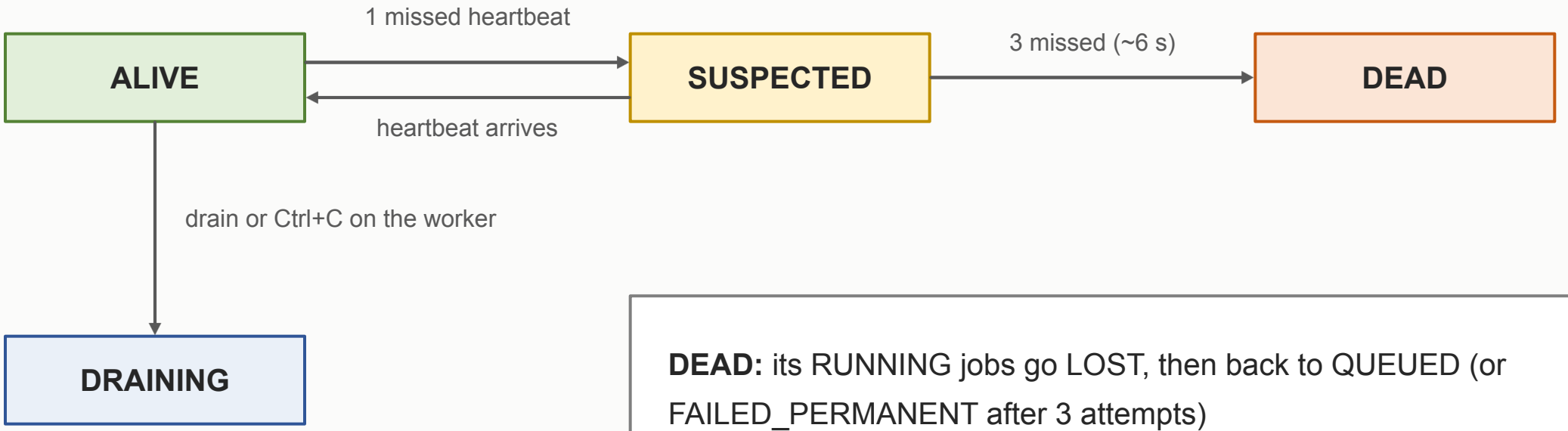
Job state machine (Section 1)



Preemption and migrate also send RUNNING back to QUEUED, without using up an attempt.

Note: You don't have to design all states at the start. This is the whole summary view

Worker state machine (Section 1)



DEAD: its RUNNING jobs go LOST, then back to QUEUED (or FAILED_PERMANENT after 3 attempts)

DRAINING: finishes its current jobs, then disconnects and exits. Gets no new jobs

Data model: what to store (Section 1)

An idea, not the only right layout.

Job (scheduler keeps one per job)

```
struct Job {
  int id; char user[32];
  char binary[256], hash[41];
  int cores, ram, gpu, time;
  int prio, eff_prio;
  time_t submit_time;
  int state; // QUEUED ...
  int worker_id, attempts;
  int deps[16], ndeps, group;
  int preemptible, hold_for;
  int cron_id, run_no;
  int avoid_worker;
  char *ckpt; int ckpt_len;
  char tail[64 * 1024];
  // live RSS, CPU, fail reason
};
```

Worker, and client session

```
struct Worker {
  int id, sock_fd;
  int state; // ALIVE ...
  int missed_hb;
  time_t last_hb;
  int cores, free_cores;
  int ram, free_ram;
  int gpu, free_gpu;
  int cpu_util, njobs;
  char cached[32][41];
};
struct Session {
  int fd; char user[32];
  int following; // logs -f
};
```

Cron template, and the rest

```
struct CronTemplate {
  int id, every, runs, active;
  time_t next_due;
  // the job fields to copy
};

// scheduler-wide state
Job jobs[MAX_JOBS];
Worker workers[100];
Session sessions[1024];
CronTemplate crons[64];
int cores_held[user];
```

Baseline

Baseline: TCP and registering (Section 2B)

Every message is a header plus a payload. Read and write loops cope with partial transfers.

One message = header + payload

```
typedef struct {
  MessageType type;
  uint32_t length; // payload bytes
} MsgHeader;

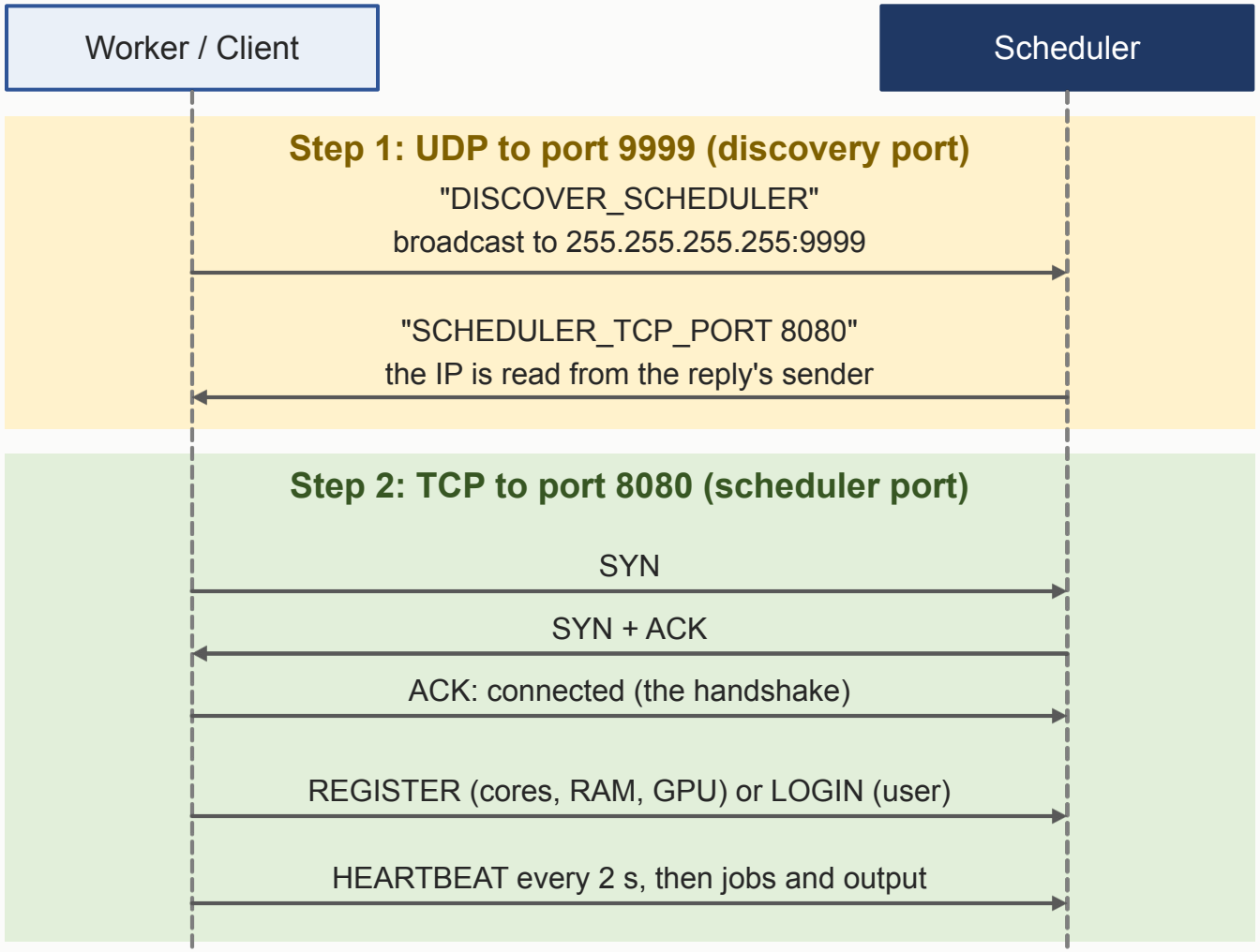
int recv_all(int fd, void *buf, size_t len) {
  size_t total = 0;
  while (total < len) {
    n = recv(fd, buf + total, len - total, 0);
    if (n == 0) return -1; // peer closed
    total += n;
  }
  return 0;
}
```

Register, then handle by type

```
// worker side
connect(sock, scheduler_ip, scheduler_port);
MsgWorkerRegister reg = { cores, ram_mb, gpu };
wsend_msg(sock, MSG_WORKER_REGISTER, &reg, ..);
// the reply carries our ID

// scheduler side, in the select() loop
new_fd = accept(tcp_fd, &cliaddr, &len);
recv_full(fd, &header, sizeof header);
recv_full(fd, payload, header.length);
switch (header.type) {
  case MSG_WORKER_REGISTER:
    // add the worker, reply with its new ID
    break;
  case MSG_CLIENT_LOGIN:
    // remember the user name
    break;
}
```

Baseline: UDP discovery, then TCP (Section 2C)



Two different ports

Discovery port (UDP)

- Default 9999, changed with -discovery-port
- Only used to ask: where is the scheduler?
- Same number on every machine, so nobody needs the scheduler's address first

Scheduler port (TCP)

- Chosen when starting it: ./scheduler 8080
  FIFO
- Carries everything real: register, jobs, heartbeats, output
- Learned from the discovery reply, or typed as the fallback ip and port

Baseline: discovery in C (Section 2C)

Scheduler: answer broadcasts

```
int udp_fd = socket(AF_INET, SOCK_DGRAM, 0);
setsockopt(udp_fd, SOL_SOCKET, SO_REUSEADDR, ..);
setsockopt(udp_fd, SOL_SOCKET, SO_REUSEPORT, ..);
bind(udp_fd, 0.0.0.0 : discovery_port);

// inside the main select() loop
if (FD_ISSET(udp_fd, &readfds)) {
  recvfrom(udp_fd, buf, ..., &cliaddr, &len);
  if (!strcmp(buf, "DISCOVER_SCHEDULER"))
    sendto(udp_fd, "SCHEDULER_TCP_PORT 8080",
        ..., &cliaddr, len);
}
```

Worker, client: ask, wait, retry

```
setsockopt(fd, SOL_SOCKET, SO_BROADCAST, ..);
dst = 255.255.255.255 : discovery_port;

for (int i = 0; i < 3; i++) {
  sendto(fd, "DISCOVER_SCHEDULER", ..., &dst);
  struct timeval tv = {2, 0};
  if (select(fd+1, &rd, NULL, NULL, &tv) > 0) {
    recvfrom(fd, buf, ..., &from, &len);
    port = atoi(buf + 19);
    ip = inet_ntoa(from.sin_addr);
    return 0;
  }
}
return -1; // caller uses the fallback ip:port
```

Core & Resilience

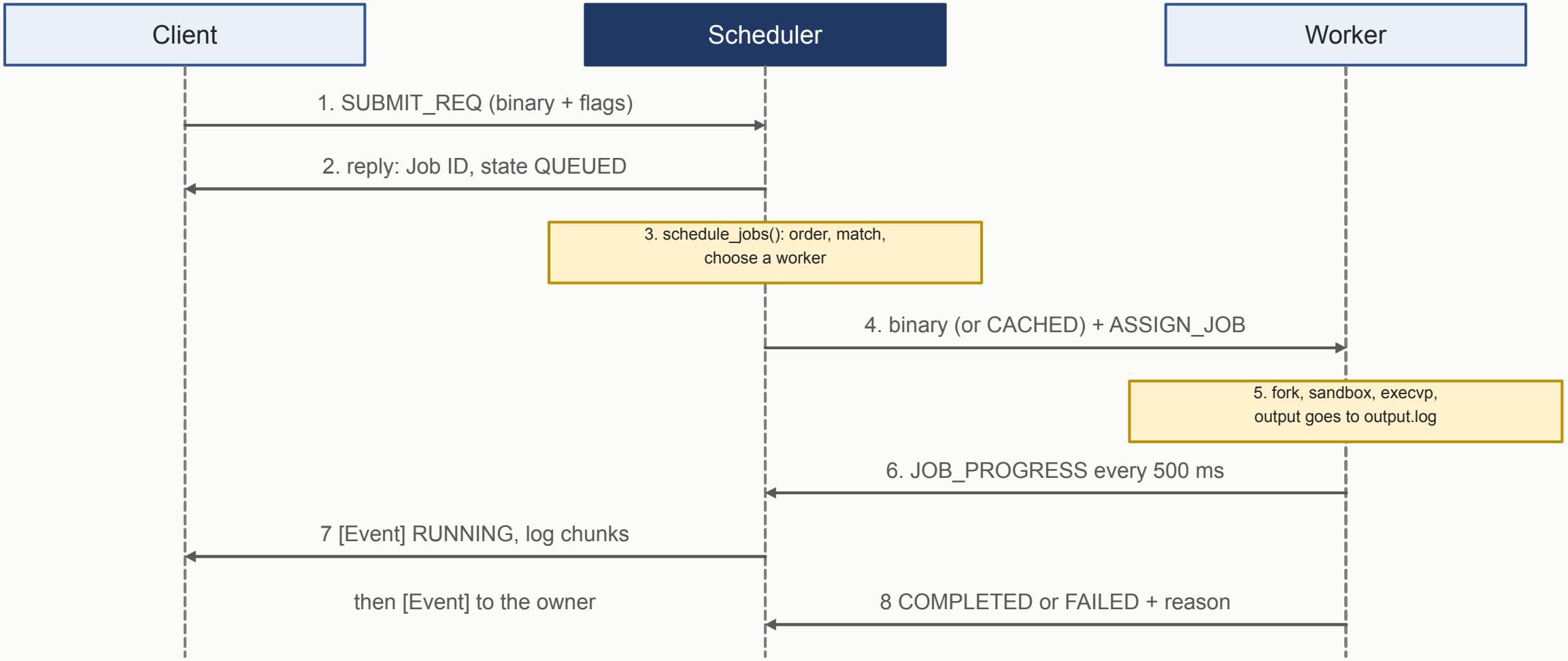
Client: syntax and commands (Section 3)

```
./client [scheduler_ip port] [-user NAME] [-discovery-port P]
```

Command	What it does	Flags
nodes	workers, state, free/total cores, RAM & GPU	none
submit BIN	create a job (or a cron template)	-cores -ram -time -prio, -gpu
status ID	state, worker, live RAM and CPU	none
logs ID [-f]	stored output, or follow it live	-f follows until Ctrl+C
cancel ID	cancel QUEUED, WAITING or RUNNING	none
migrate ID	move a RUNNING job elsewhere	none
crons	list recurring templates	cancel-cron ID stops one

Extra submit flags: **-group G**, **-preemptible**, **-every S** (cron), **-depends-on 1,2,g3** (jobs 1 and 2 plus every job in group 3 at submit time; unknown IDs are rejected).

Job submit pipeline (Sections 3, 4B, 5)



Note: the client sends only the path to the binary. The scheduler reads the file itself and sends its bytes to the worker.

Scheduler: its tasks (Section 4A)

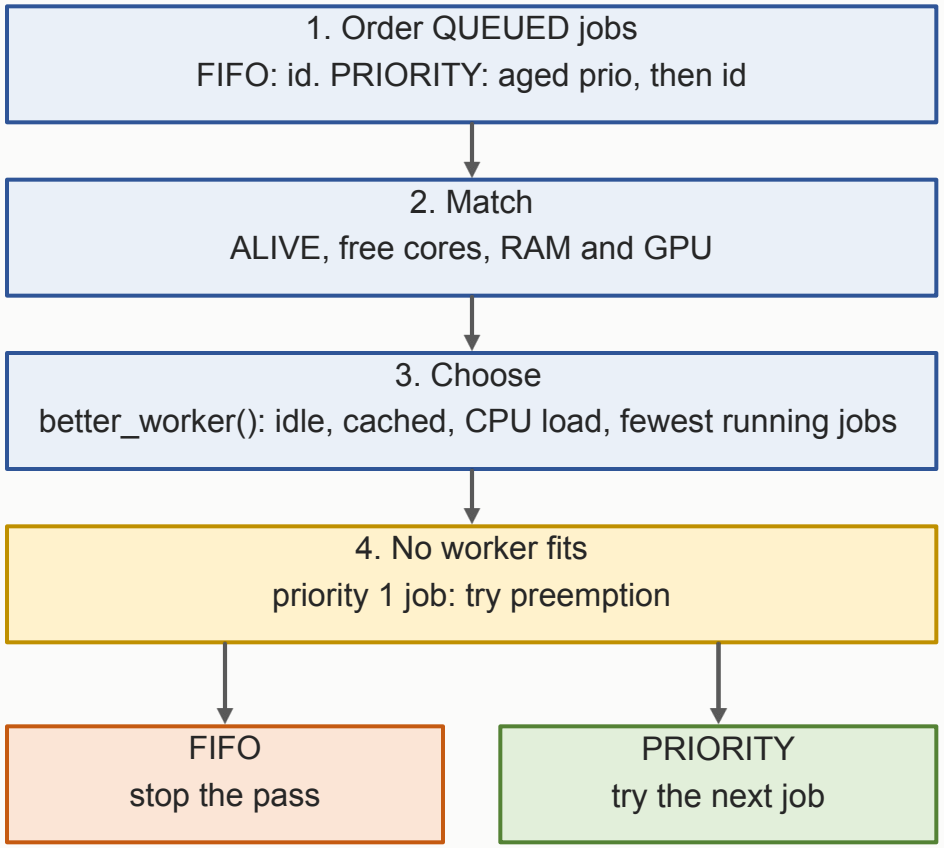
- Answer UDP discovery and accept TCP connections
- Register workers, log clients in
- Handle client requests: submit, nodes, status, logs, cancel
- Run the scheduling pass and track heartbeats
- Relay job output and [Event] lines to clients

One thread, one select() loop: no locks, and nodes or status never wait behind a scheduling decision. The 1 s timeout guarantees a tick even when the network is quiet.

```
// the scheduler's main loop, simplified
while (1) {
  struct timeval tv = {1, 0};
  select(max_fd + 1, &readfds, &writefds, NULL, &tv);

  if (now - last_schedule >= 1) {
    cron_tick();
    schedule_jobs(algorithm);
    check_heartbeats();
    end_finished_follows();
  }
  if (FD_ISSET(udp_fd, &readfds)) {
    // answer the discovery request
  }
  if (FD_ISSET(tcp_fd, &readfds)) {
    // accept the new connection
  }
  for (int i = 0; i < nfds; i++) {
    // read the header, then the payload
    // handle it by header.type
    // e.g. a submit calls schedule_jobs()
  }
}
```

Scheduling: the basic version (Section 4B)




```
void schedule_jobs(const char *algo) {
  // aging (PRIORITY only): prio-- every 10 s waiting
  // sort the QUEUED jobs into sorted_idxs[]
  for (int i = 0; i < queued_count; i++) {
    Job *j = &jobs[sorted_idxs[i]];
    best_w = -1;
    for (int w = 0; w < num_workers; w++) {
      // skip if not ALIVE or j does not fit
      if (best_w == -1 || better_worker(w, best_w))
        best_w = w;
    }
    if (best_w == -1 && j->prio == 1)
      try_preempt_for(j);
    if (best_w != -1) {
      j->state = RUNNING;
      w->free_cores -= j->cores;
      dispatch_job(j, w);
    } else if (is_fifo) break; // the oldest job blocks the
rest
  }
}
```

Start with this. The full version, with every feature, is on the next slide.

The full scheduling pass, in order (Section 4B)

Before looking at any job

1. QUEUED jobs only. WAITING jobs are skipped until their dependencies COMPLETE

↓

2. Aging (PRIORITY only): a QUEUED job moves up one priority level every 10s, until it reaches 1

↓

3. Sort. FIFO: submission order. PRIORITY: effective priority, then older first

Then, for each job in that order

4. Preempted-job hold: skip it until the priority 1 job that took its place has started

↓

5. Quota: skip it if its user holds over 50% of the cores and another user (not over quota) has a QUEUED job

↓

6. Match: ALIVE (not DRAINING) with enough cores, RAM, GPU. Leave out the worker it was just migrated off

↓

7. Choose: idle first, then binary cached, then lowest CPU%, then fewest jobs

↓

8. No match: a priority 1 job tries preemption. Then FIFO stops the pass, PRIORITY moves on

After a worker is chosen

9. Send the saved checkpoint, if the job has one

↓

10. Send the binary, or the short "already cached" message

This is the basic version from the last slide with every later feature added.

Runs on submit, job finish, new worker, heartbeat, and at least once a second.

At first you only need steps 2, 3, 6 and 8, and step 7 can take the first worker that fits. The others are filled in as you add features.

Heartbeats: detect and rescue (Section 4C)



Heartbeat arrives: back to ALIVE. DEAD: its RUNNING jobs go LOST, then QUEUED again (or FAILED_PERMANENT after 3 attempts).

Worker: send every 2 s

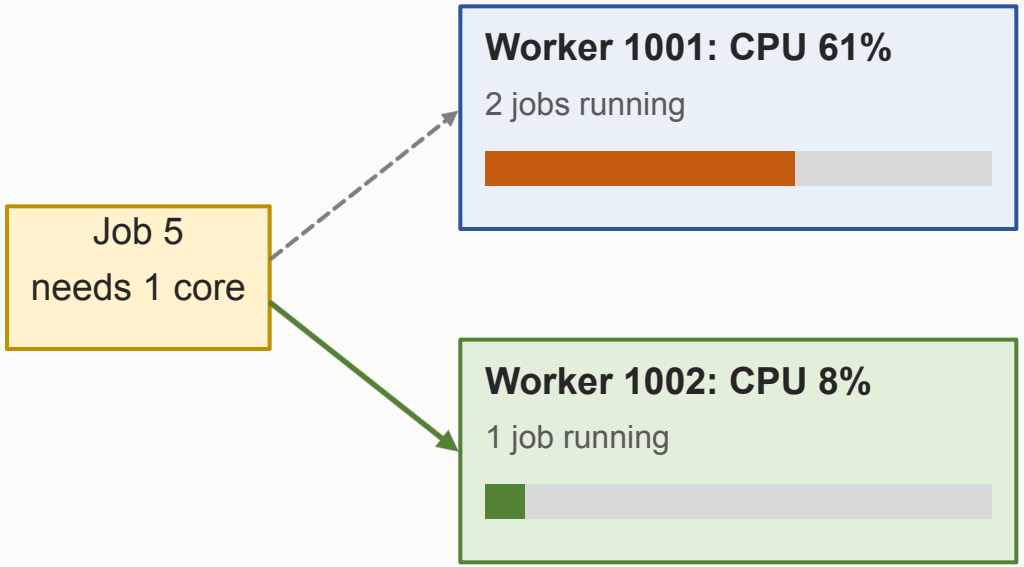
```
if (now - last_heartbeat >= 2) {
  getloadavg(load, 1);
  cpu_pct = load[0] * 100 / total_cores;
  MsgWorkerHeartbeat hb = { worker_id,
    cpu_pct, free_ram, num_active,
    free_cores };
  wsend_msg(sock, MSG_WORKER_HEARTBEAT,
    &hb, sizeof hb);
  // then send one JOB_STATS per running job
}
```

Scheduler: record, then check

```
// when a HEARTBEAT arrives
w->last_heartbeat = now_sec();
w->cpu_util = hb->cpu_util_pct;
if (w->state == SUSPECTED) w->state = ALIVE;

// check_heartbeats(), runs every second
age = now - w->last_heartbeat;
if (age > 6.0) {
  w->state = DEAD;
  for (int k = 0; k < num_jobs; k++) {
    // for each job RUNNING on w:
    // 3rd attempt used: FAILED_PERMANENT
    // otherwise: LOST, then QUEUED again
  }
} else if (age > 2.5) w->state = SUSPECTED;
```

Load-aware scheduling (Section 4D)

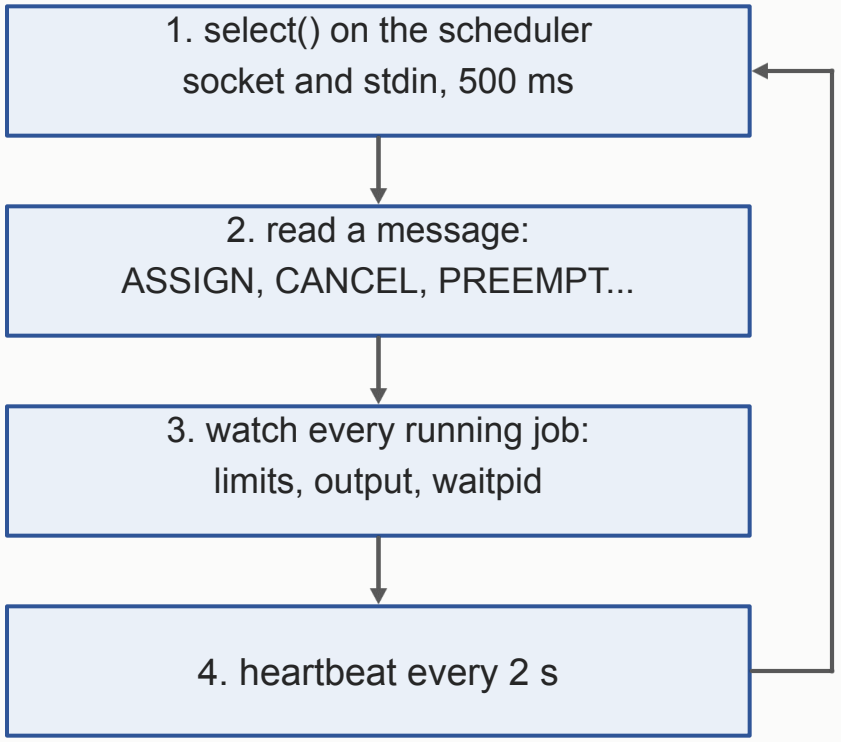


Both workers have free cores. The less busy one wins, and the decision is logged.

```
// order: idle, cached, lowest CPU, fewest jobs
static int better_worker(const Worker *x,
    const Worker *y, const char *hash) {
  int xi = x->active_jobs == 0;
  int yi = y->active_jobs == 0;
  if (xi != yi) return xi;
  if (hash && worker_has(x, hash) != worker_has(y, hash))
    return worker_has(x, hash);
  if (x->cpu_util != y->cpu_util)
    return x->cpu_util < y->cpu_util;
  return x->active_jobs < y->active_jobs;
}
```

- CPU load = getloadavg() / cores x 100, capped at 100, sent in every heartbeat
- Log: Job 5 -> Worker 1002 (chosen over 1001: CPU 8% vs 61%)

Worker: how it works (Section 5)



Capacity (cores, RAM) is clamped to the real machine.

```
// the worker's main loop, simplified
while (!stop_requested) {
  select(maxfd + 1, &readfds, NULL, NULL, &tv);
  if (FD_ISSET(STDIN_FILENO, &readfds)) {
    // if the line says "drain":
    is_draining = 1;
    wsend_msg(sock, MSG_WORKER_DRAIN, ..);
  }
  if (FD_ISSET(sock, &readfds)) {
    // read the header and payload, then by type:
    // ASSIGN_JOB: launch_job(), free -= job
    // CANCEL_JOB: kill_job(pid, SIGKILL)
  }
  for (int i = 0; i < num_active; i++) {
    // check the limits, send new output
    if (waitpid(pid, &status, WNOHANG) > 0) {
      // send COMPLETED or FAILED, free resources
    }
  }
  // every 2 s: send a heartbeat
}
```

Limiting a job: what to do? (Section 5A)

> This is not a real sandbox: no containers, namespaces or chroot. We only use plain POSIX calls to limit the damage a job can do.

Goal	How we enforce it	Calls
Own files	job dir /tmp/osn_job_ID, own copy of the binary	mkdir, chdir, chown
No privileges	run as nobody (worker started with sudo)	setgid, setuid
CPU time	CPU limit plus a wall-clock timer	setrlimit, time
Memory	address-space limit, RSS check on macOS	setrlimit, proc_pidinfo
Forks, disk	process and file-size limits, close inherited fds	setrlimit, close
Output	stdout and stderr go to output.log	dup2

Only -time and -ram are enforced by the OS. -cores and -gpu are scheduler bookkeeping.

Limits (1): directory and user (Section 5A)

- Each job gets its own folder with a copy of the binary, so it can't touch your Desktop or home folder
- Run chown before setuid: after dropping root you can no longer change the folder's owner
- Drop the group first, then the user (setgid, then setuid). If either fails, don't run the job
- Not root? Warn and run without dropping privileges, so you can develop without sudo
- Put the job in its own process group, so one kill stops it and everything it started

```
// launching a job, simplified
mkdir("/tmp/osn_job_ID", 0755);
if (geteuid() == 0) { // started with sudo
  pw = getpwnam("nobody");
  chown(dir, pw->pw_uid, pw->pw_gid);
  copy_binary_into(src, dir);
}
pid = fork();
if (pid == 0) { // child process
  setpgid(0, 0);
  chdir(dir);
  dup2(fd_out, STDOUT_FILENO); // to output.log
  dup2(fd_out, STDERR_FILENO);
  for (fd = 3; fd < fdmax; fd++) close(fd);
  // drop privileges;
  if (setgroups(0, NULL) < 0 ||
    setgid(gid) < 0 || setuid(uid) < 0)
    exit(126);
  // then setrlimit CPU, RAM, NPROC (next slide)
}
```

Limits (2): resource limits (Section 5A)

```
// still in the child, after dropping privileges, before execvp()
struct rlimit cpu = { time_sec, time_sec };
setrlimit(RLIMIT_CPU, &cpu); // kernel sends SIGXCPU

struct rlimit mem = { ram_mb * 1024 * 1024,
    ram_mb * 1024 * 1024 };
setrlimit(RLIMIT_AS, &mem); // ignored on macOS

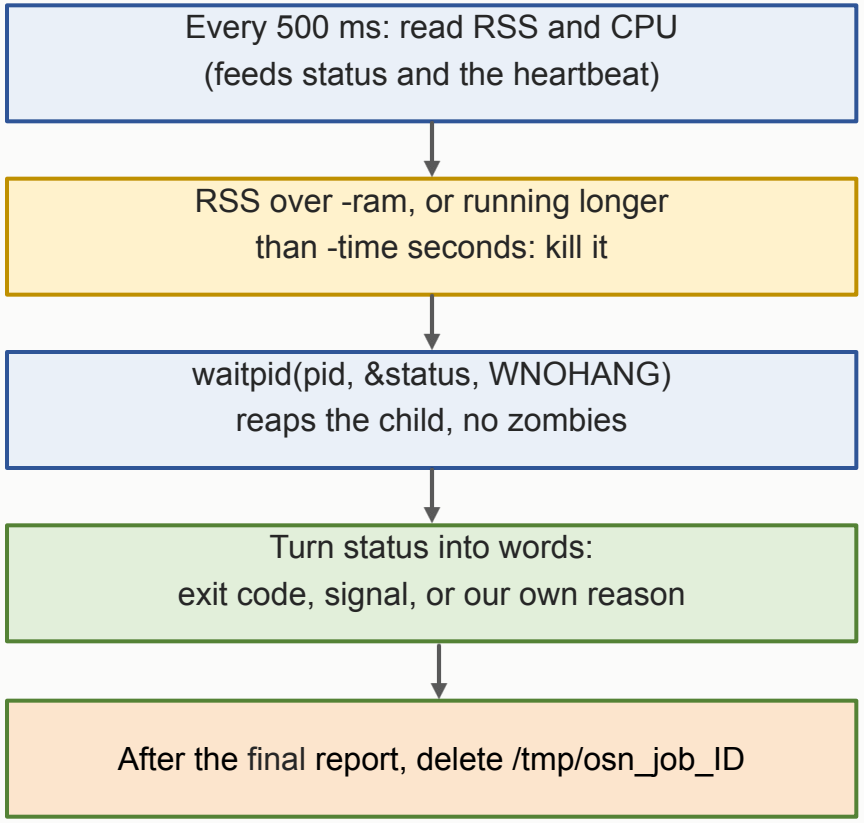
struct rlimit fsz = { 100 << 20, 100 << 20 };
setrlimit(RLIMIT_FSIZE, &fsz); // caps the log size

// after setuid(): this limit is per user
struct rlimit np = { 64, 64 };
setrlimit(RLIMIT_NPROC, &np);

execvp(args[0], args);
exit(127); // exec failed: command not found
```

- RLIMIT_CPU: kernel sends SIGXCPU past -time of CPU
- RLIMIT_AS: allocation fails past -ram
- RLIMIT_NPROC is counted per user and every job is nobody, so all jobs share one budget. Too small and a job that forks fails at once: use about 64
- Exit code 127 is how the worker later explains "command not found"

Limits (3): watching (Sections 5A, 5D, 5E)




```
job_telemetry(wj->pid, &rss, &flt, &cpu);
// reads /proc/PID/stat on Linux, proc_pidinfo on macOS
if (rss > wj->ram_mb) {
  wj->kill_reason = 2; // memory limit
  kill_job(wj->pid, SIGKILL);
} else if (time(NULL) - wj->start >= wj->time_sec) {
  wj->kill_reason = 1; // wall-clock limit
  kill_job(wj->pid, SIGKILL);
}

// diagnose_status(): explain how it ended
if (WIFSIGNALED(status)) sig = WTERMSIG(status);
else if (WIFEXITED(status)) code = WEXITSTATUS(status);
// code 127 means: could not execute the binary
if (wj->kill_reason == 1)
  diag = "Wall-clock timeout exceeded (Ns limit)";
```

RSS : resident set size (amount of real RAM a process is using right now)

How to test the limits (Section 5)

Write each test as a tiny C program (about 5 lines) and submit it. Run the worker with sudo.

Submit a job that...	You should see
writes a file in your home folder	Permission denied in its output (the privilege drop works)
loops forever, with -time 3	FAILED: Wall-clock timeout exceeded (3s limit)
sleeps longer than -time	The same message. CPU limits never fire for a sleeper, the timer does
keeps allocating, with -ram 100	Killed, with a memory-limit reason. status shows RAM climbing
writes to a null pointer	Terminated by signal 11 (SIGSEGV)
divides by zero	Terminated by signal 8 (SIGFPE)
exits with code 3, or a bad command name	Exited with code 3, or code 127 (command not found)
forks in a loop	Stops at the process limit (all jobs share one budget: test it alone)
runs long, then you run cancel	CANCELLED, resources freed, no zombie left in ps

Rules that are easy to miss: core system

Worker IDs (2B)
The scheduler gives each worker its ID in the registration reply. Before that, the worker logs without an ID.

Connection lost (2B)
The worker kills its running jobs, finds the scheduler again and registers as a NEW worker with a NEW ID.

Discovery (2C)
Run it at startup and again whenever the connection drops. Use the typed ip:port only after every retry has failed.

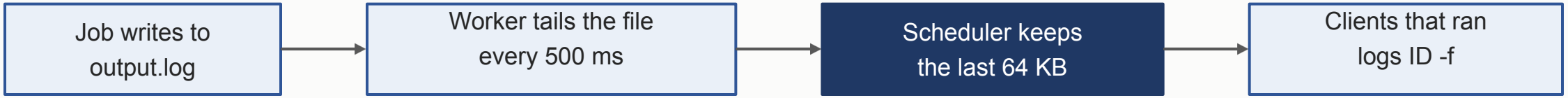
Client login (3)
Name: 1 to 31 chars of letters, digits, _ - . else ask again. -user skips the prompt. After a reconnect the client logs in again by itself. No password.

Full queue (4A)
Max 1024 queued jobs. Then submit answers QUEUE_FULL and the client says so. Never block, never lose a job silently. Jobs the scheduler requeues itself are never dropped.

Logs name the user (3)
Every scheduler log line about something a client did says which user did it.

Additives

Additive 1: streaming logs (Section 6)



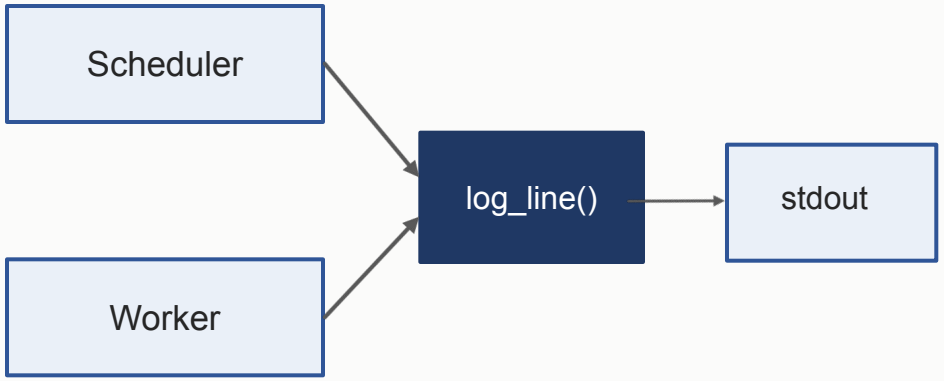
Worker: send what is new

```
// for each running job, every loop
while ((n = read(wj->log_fd, buf, 4096)) > 0) {
  MsgWorkerJobProgress prog =
    { wj->job_id, n };
  // send the PROGRESS header, then prog,
  // then the n bytes of text
}
// log_fd is O_NONBLOCK: read() returns
// 0 when there is nothing new
```

Scheduler: store and forward

```
case MSG_WORKER_JOB_PROGRESS:
  // append to j->log_buffer, keep the last 64 KB
  for (int k = 0; k < 1024; k++)
    if (client_subscriptions[k] == job_id)
      send_msg(client_sockets[k],
        MSG_CLIENT_LOG_CHUNK, data, len);
  break;
case MSG_CLIENT_LOGS_REQ:
  // send the stored tail first
  if (follow && job_not_finished)
    client_subscriptions[i] = job_id;
  break;
```

Additive 2: logging (Section 7)

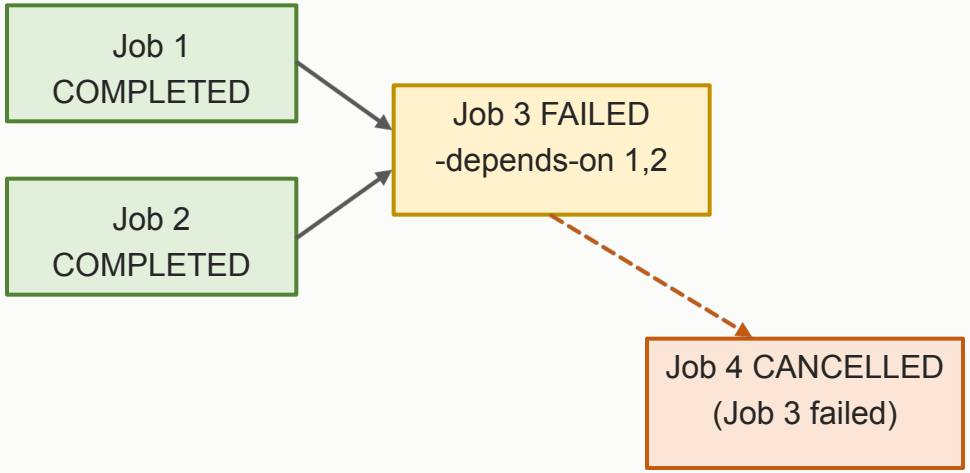


- Log every state change, scheduling decision, discovery outcome

```
// one shared function, used everywhere
void log_line(const char *format, ...) {
  time_t t = time(NULL);
  localtime_r(&t, &tm);
  strftime(time_str, sizeof time_str,
    "[%H:%M:%S]", &tm);
  vsnprintf(body, sizeof body, format, args);
  snprintf(line, sizeof line, "%s %s\n",
    time_str, body);
  fwrite(line, 1, n, stdout);
  fflush(stdout);
}

// usage
log_line("[SCHEDULER] Worker %d -> DEAD (3 missed
  heartbeats)", w->id);
```

Additive 3: dependencies, groups (Section 8)

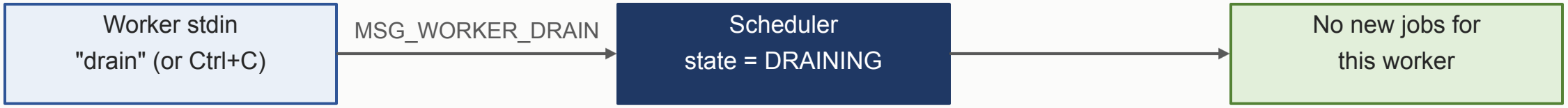


- g3 is resolved at submit time into the job IDs in group 3 at that moment. Jobs that join later don't count.
- A failed dependency cancels waiters, and it cascades

```
// every dep is a job id (groups are expanded at submit)
int dep_status(waiter, dep, &culprit) {
  if (dep > 0) {
    st = jobs[dep - 1].state;
    if (st == COMPLETED) return 1;
    if (st == FAILED || st == CANCELLED ||
      st == FAILED_PERMANENT) return -1;
    return 0; // still pending
  }
}

void check_dependencies() { // every pass
  do {
    changed = 0;
    for (int i = 0; i < num_jobs; i++) {
      // for each WAITING job w, ask dep_status():
      // any -1: w->state = CANCELLED
      // all 1: w->state = QUEUED
    }
  } while (changed); // cascades down chains
}
```

Additive 4: graceful draining (Section 9)



Meanwhile the worker keeps heartbeating, finishes its running jobs, sends their output, then exits. A drained worker leaving is not a failure: nothing is retried.

Worker

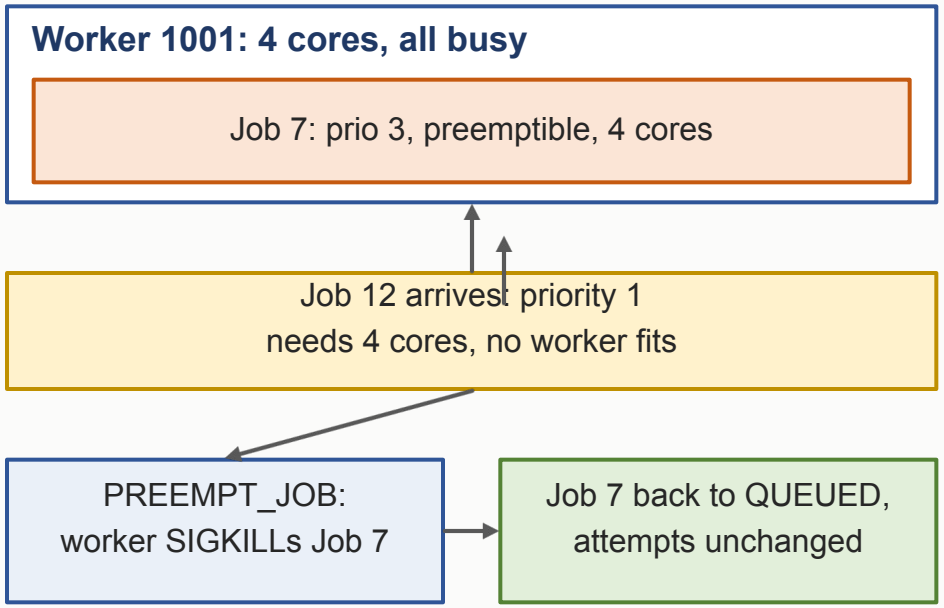
```
if (!strncmp(buf, "drain", 5) && !is_draining) {
  is_draining = 1;
  wsend_msg(sock, MSG_WORKER_DRAIN, NULL, 0);
}
// Ctrl+C (SIGINT) takes this same path: the handler
sets a flag
if (is_draining && num_active == 0) {
  log_line("... Drain complete. Exiting.");
  break;
}
```

Scheduler

```
case MSG_WORKER_DRAIN:
  w->state = DRAINING;
  log_line("Worker %d -> DRAINING", w->id);
  broadcast_event("Worker is now DRAINING.");
  break;

// matching only takes ALIVE workers, so a
// DRAINING one gets nothing new. A heartbeat
// must not flip it back to ALIVE.
```

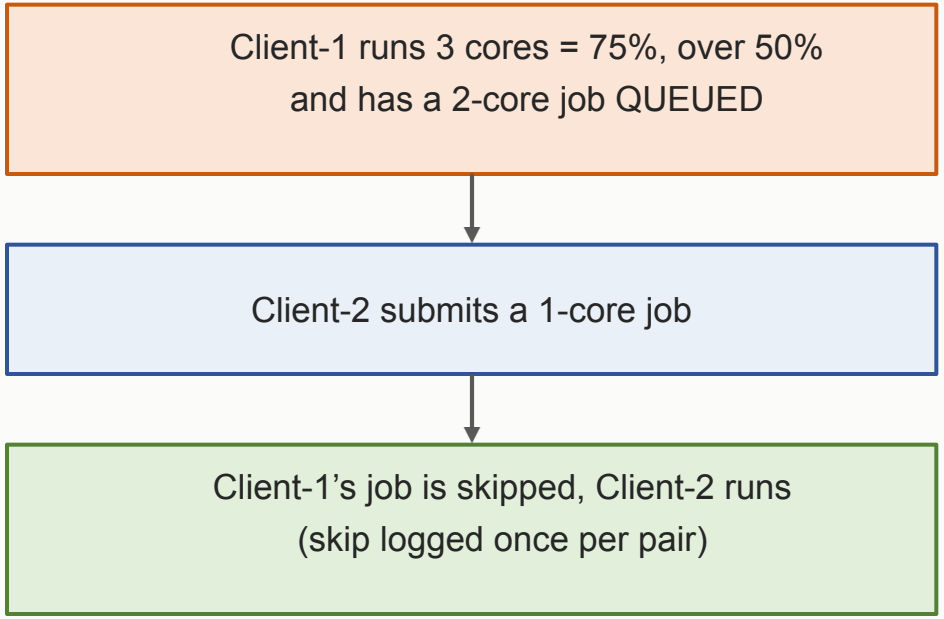
Additive 5: preemptible jobs (Section 10)




```
// try_preempt_for(j): j is priority 1
for (int w = 0; w < num_workers; w++) {
  // wk = &workers[w], ALIVE workers only
  fc = wk->free_cores; fr = wk->free_ram;
  // cand[] = jobs RUNNING on wk, preemptible, prio > 1
  // sort cand[]: highest prio number first, then newest
submit first
  for (int c = 0; c < ncand; c++) {
    v = cand[c];
    victims[nv++] = v;
    fc += v->cores; fr += v->ram_mb;
    if (fc >= j->cores && fr >= j->ram_mb) break;
  }
  if (fc < j->cores || fr < j->ram_mb)
    continue; // not enough even then
  for (int n = 0; n < nv; n++) { // each victim v
    v->preempting = 1;
    v->defer_for = j->id; // hold until j runs
    send_msg(wk->sock_fd,
      MSG_WORKER_PREEMPT_JOB, &v->id, ..);
  }
  return;
}
```

Additive 6: fair-share quotas (Section 11)

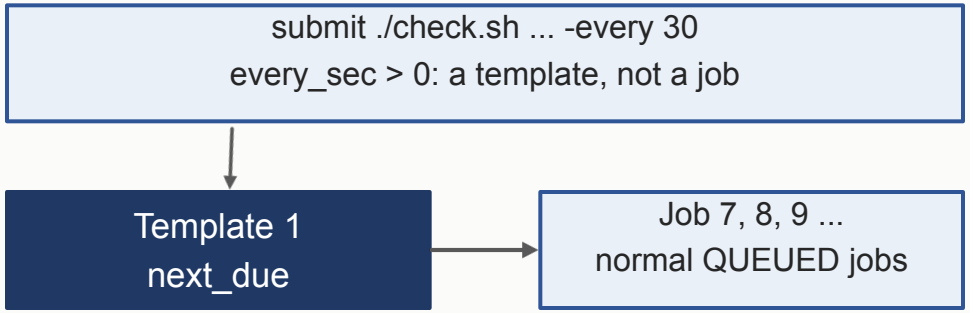
One 4-core worker




```
static int user_held_cores(const char *user) {
  for (int i = 0; i < num_jobs; i++) {
    // if job i is RUNNING, holding cores
    // and belongs to this user:
    c += jobs[i].cores;
  }
  return c;
}
static int user_over_quota(const char *user) {
  // MORE than 50% of all non-dead cores
  return user_held_cores(user) * 2 >
       total_cluster_cores();
}

// in schedule_jobs(), before matching workers
if (user_over_quota(j->user)) {
  // fav = first QUEUED job of another user
  // who is not over quota
  if (fav) {
    // log "Skipping Job ..." once
    continue;
  }
}
```

Additive 7: Cron jobs (Section 12)



- First copy runs at once, then every N s
- cancel-cron sets active = 0: future copies only

```
// handle_cron_submit(): register the template
t->id = next_template_id; t->every = every_sec;
t->active = 1; t->next_due = now_sec();

static void cron_tick(void) {
  for (int i = 0; i < num_crons; i++) {
    CronTpl *t = &crons[i];
    if (!t->active || now < t->next_due)
      continue;
    t->next_due += t->every;
    cron_fire(t);
  }
}
static void cron_fire(CronTpl *t) {
  jid = create_job(t->binary, t->cores, ...);
  jobs[jid - 1].cron_tpl = t->id;
  t->runs++;
  // tell the owner: "Cron %d injected Job %d (run #%d)"
}
```

Rules that are easy to miss: additive jobs

Preemption (10)
Kill only as many preemptible jobs as needed. Victims: lowest priority first (highest number); among equals, the most recently submitted first. A victim is not retried until the priority 1 job has STARTED. If that job is cancelled or fails first, release the hold. No attempt is used.

Draining (9)
"drain" on the worker stdin and Ctrl+C (SIGINT) must do exactly the same thing. There is no client command for it, on purpose. A drained worker leaving is not a failure: nothing is retried.

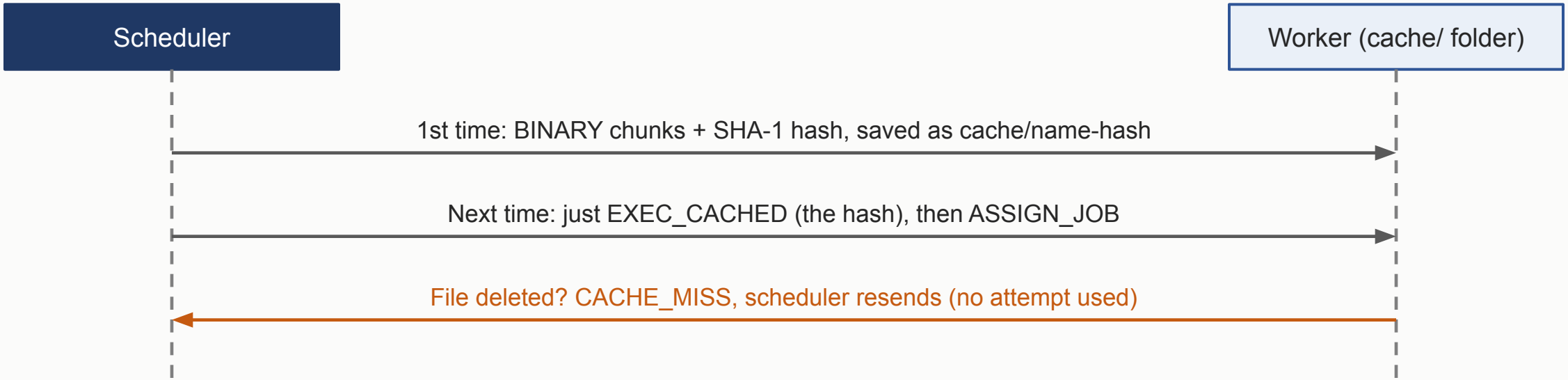
Cron (12)
-every cannot be combined with -depends-on. Template ids are numbered separately from job ids, starting at 1. The first copy runs at once, then one every N s, even if the last copy is still running. cancel-cron stops FUTURE copies only. An unknown or already cancelled id gives a clear error.

And finally the hard parts :(

Hard part 1: executable caching (Section 13)

The problem
Suppose a client submits the same 30 MB program 40 times. A plain scheduler sends all 30 MB to a worker, 40 times.

The idea
Send a program to a worker once. The worker keeps a copy (a cache), so next time the scheduler only says "you already have this one".



A program is identified by its content (SHA-1), not its name. Same name with different content counts as a different program.

Caching: which worker, edge cases (Section 13)

Which worker gets the job?

1. Idle (no running jobs) comes first

↓

2. Then the one that has the file cached

↓

3. Then the lowest CPU load

↓

4. Then the fewest running jobs

An idle worker without the file still beats a busy worker that has it: spreading work to a free machine matters more than saving one transfer. A worker that cannot fit the job is never considered at all.

Cases you must handle

Worker restarts
On reconnect it scans its cache/ folder and tells the scheduler which hashes it still has.

A cached file has disappeared
The worker answers CACHE_MISS. The scheduler forgets that hash for that worker and sends the full file. It does not count as a failed attempt.

The scheduler cannot read the program
For example ls, a command found through PATH. No caching and no transfer: the worker runs it by name.

Caching (1): the scheduler side (Section 13)

- exec_info() hashes the file with SHA-1 (and remembers the result by mtime)
- Per worker we keep cached[32][41], the hashes it holds
- Known hash: send a tiny EXEC_CACHED. Unknown: stream the file, then remember it
- A PATH command like ls is not a file: no transfer, the worker runs it by name
- CACHE_MISS from a worker: forget that hash for it and resend the whole file. Not a failed attempt

```
// dispatch_send(j, w)
if (exec_info(j->binary, path, .., &size, hash)) {
  if (worker_has(w, hash)) {
    send_msg(fd, MSG_WORKER_EXEC_CACHED, hash);
    msg.exec_mode = 1;
    // copy the hash into msg.exec_hash
  } else if (send_binary(w, path, hash, size) == 0) {
    worker_add_cache(w, hash);
    msg.exec_mode = 1;
  }
}
send_msg(fd, MSG_WORKER_ASSIGN_JOB, &msg, ..);

// send_binary(): 48 KB chunks
while (off < size) {
  n = fread(chunk, 1, 49152, f);
  // header = { hash, name, total, offset, len }
  send_msg(fd, MSG_WORKER_BINARY, header + chunk);
  off += n;
}

// on MSG_WORKER_CACHE_MISS from worker w
worker_drop_cache(w, hash); // forget it
// dispatch_send() again: full file, attempts unchanged
```

Caching (2): the worker side (Section 13)

LRU with a 2 MB limit



Three programs of about 0.7 MB each. Submit A, B, A, C. A was used after B, so when C arrives B is evicted, not A. B again needs a full transfer.

- Never evicted: the file just stored, and any file a running job uses
- Limit: OSN_CACHE_SIZE_MB (default 512)
- Own folder per worker: OSN_CACHE_DIR
- The job runs a copy placed in its own job folder

```
// receiving MSG_WORKER_BINARY chunks
// append each chunk to a temp file
if (last chunk && sha1(tmp) == hash) {
  rename(tmp, "cache/name-hash");
  cache_evict(hash);      // keep the new file
  send_cache_list(sock);  // tell the scheduler what is left
}

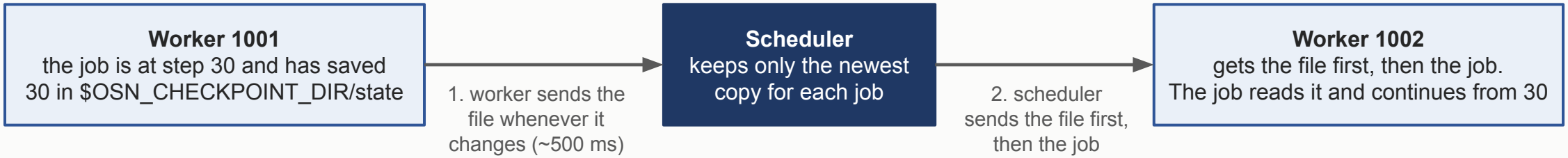
// on ASSIGN_JOB with exec_mode == 1
if (cache_lookup(job->exec_hash, cpath, ..)) {
  utimes(cpath, NULL); // marks it used now (LRU)
  // the sandbox copies cpath into the job folder
} else {
  // tell the scheduler: MSG_WORKER_CACHE_MISS
}

// cache_evict(keep_hash)
while (total > limit) { // OSN_CACHE_SIZE_MB
  // victim = oldest mtime, not keep_hash,
  // and not cache_hash_in_use()
  if (victim == -1) break;
  unlink(victim_path);
  total -= victim_size;
}

// after (re)connecting
send_cache_list(sock); // scans cache/ for hashes
```

Hard part 2: checkpoints (Section 14)

What is a checkpoint? A small file where the job saves how far it got, like saving a game. If the job is moved or its worker dies, the next worker hands the file back and the job carries on from there instead of starting from zero.



- **The job does the saving.** It writes a file called state inside the folder named by $OSN_CHECKPOINT_DIR
- At startup it looks for that file. **Found:** continue from it. **Not found:** start from the beginning
- Write state.tmp first, then rename() it to state, so nobody ever reads half a file
- **The worker only copies the file** to the scheduler. **The scheduler keeps just the newest one** per job, in memory (a few KB, no shared filesystem)
- Used again after a migrate, a preemption or a dead worker. A job that never saves just restarts from scratch

```
// the example job: ./counter N
int i = 1;
path = $OSN_CHECKPOINT_DIR + "/state";
f = fopen(path, "r");
if (f) fscanf(f, "%d", &i); // resume
for (; i <= N; i++) {
  printf("step %d\n", i); fflush(stdout);
  sleep(1);
  if (i % 5 == 0) {
    // write i to path + ".tmp", then:
    rename(path + ".tmp", path);
  }
}
```

Checkpoints: the platform side (Section 14)

Worker: prepare, copy, restore

```
// 1. before the job starts
mkdir(job_dir + "/checkpoint");
// delete any old state file
// if the scheduler sent a saved checkpoint,
//   write it to checkpoint/state now
chown(dir, nobody);  // only if root, so the
                     // job can write there
setenv("OSN_CHECKPOINT_DIR", dir, 1);

// 2. while it runs, every ~500 ms
stat(state_path, &st);
sig = mtime ^ inode ^ size; // fingerprint
if (sig == wj->ckpt_sig) return; // unchanged
// changed: read the file (a few KB) and
wsend_msg(sock, MSG_WORKER_CHECKPOINT, buf, ..);
wj->ckpt_sig = sig;

// 3. migrate: SIGTERM, SIGKILL after 2 s
// when the job has exited, read state ONE
// more time and send it, THEN report the
// job stopped. Otherwise the last save is lost
```

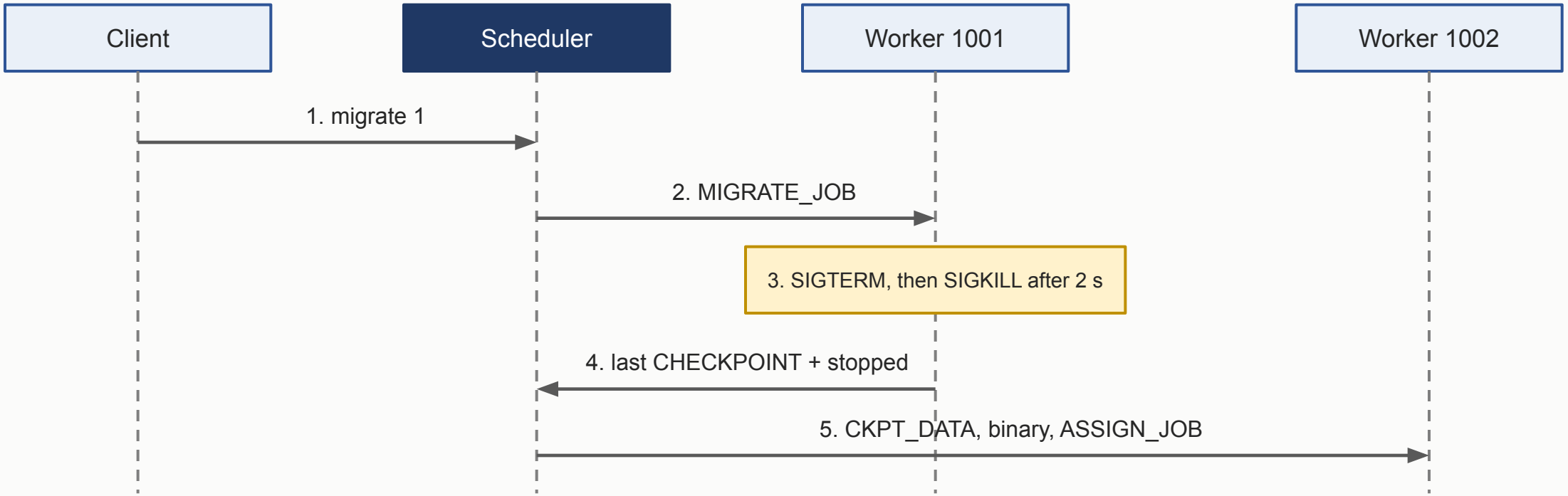
Scheduler: keep, and send first

```
// 1. a checkpoint arrives from a worker
case MSG_WORKER_CHECKPOINT:
  // replace the stored bytes with the new ones
  j->ckpt_n++;  j->ckpt_at = now;
  break;

// 2. the job is scheduled again (after a
//    migrate, a preemption or a dead worker)
// ... a worker is chosen, then send, in order:
//   a) the saved checkpoint, if there is one
if (j->ckpt && j->ckpt_len > 0)
  send_msg(w->sock_fd, MSG_WORKER_CKPT_DATA, ..);
//   b) the binary, or EXEC_CACHED
//   c) ASSIGN_JOB
send_msg(.., MSG_WORKER_ASSIGN_JOB, ..);

// 3. status ID shows: checkpoint #n, size, age
```

Live migration: migrate ID (Section 14)



1. The client asks for the move. Job not RUNNING or unknown id: print a clear error

2. The scheduler tells Worker 1001 to stop job 1

3. The worker sends SIGTERM so the job can save one last checkpoint. Still alive after 2 s: SIGKILL

4. The worker sends that last checkpoint, then reports the job stopped. The job goes back to QUEUED and no retry attempt is used

5. The next pass picks a worker (skips 1001 if any other fits). The scheduler sends the checkpoint, then the binary (or EXEC_CACHED), then ASSIGN_JOB. The job reads state and continues from about step 30

How to test migration (Section 14)

1. **Setup.** Start a scheduler and two workers. Submit a counting job: it prints step 1 to N, sleeps 1 s per step and saves its step every 5 steps

2. **Migrate.** Around step 30, run: migrate ID. The client says which checkpoint the job will resume from

3. **Check.** Job 1 should now run on the other worker and continue from about step 30, not from step 1

4. **Crash test.** Repeat, but kill the worker running it. After the dead-worker timeout (~6 s) it resumes the same way, from the last checkpoint it saved. A kill gives no last save, so it may be a few steps behind

5. **Errors.** status ID shows the latest checkpoint: number, size, age. migrate on an unknown or finished job must print a clear error

```
client> migrate 1
[Client] Migrating Job 1 off Worker 1001.
  It will resume from checkpoint #6.

[Event] Job 1 stopped on Worker 1001 and
  moved back to QUEUED; it resumes
  from checkpoint #6.
[Event] Job 1 RUNNING on Worker 1002.

client> logs 1
step 28
step 29
step 30  <- ran on Worker 1001
step 30  <- continues on Worker 1002
step 31
...
```

Questions?

Distributed Compute Cluster | OSN Course Project